CS3230 HW1

BY RAMON BESPINYOWONG (MS.) A0088687A

S1 [TWO FUNDAMENTAL PROCESSES IN ALL OF CS]

(A) (5 POINTS) [SHOCK YOUR FRIENDS – GUESS THE NUMBER GAME.]

The problem in this question is the search problem. The method to be used in this question is binary search algorithm, which is such an algorithm that it continuously cut half of all the possible values. Firstly, it selects the middle number, if the number is too low, it will search in the higher range of number; otherwise, it will search in the lower range of number. Below is the pseudo code for binary search.

```
Function binarysearch(sortedArray, start, end, I, value):
        Mid = (start + end) / 2

If start > end:
        Return -1

If sortedArray.at(mid) is equal to value:
        Return mid
Else If sortedArray.at(mid) is less than value:
        Return binarySearch(sortedArray ,mid+1, end, value)
Else:
        Return binarySearch(sortedArray, start, mid-1, value)
```

This algorithm can make Steven find the correct number as shown in the proof below.

Proof by induction

Base case: the array is empty. Therefore, n = end – start + 1 = 0. Hence, start = end + 1 and start > end. So, the function cannot find the element, which is true.

Inductive Step: Show that the binary search works for an array, of which size is n = end – start + 1 > 0.

Inductive hypothesis: Assume that the function returns the correct value for all n' such that 0 <= n' <= n-1 where n' = end'-start'+1.

(1) Mid = (start + end) / 2

(2) Therefore, start <= Mid <= end
(3) If value is equal to sortedArray[mid]
    i. The algorithm returns mid.
    ii. The algorithm works correctly.
(4) If value is less than sortedArray[mid]
    i. The value exists in the array if and only if it is existed in sortedArray[start..mid-1] because the array is sorted.
    ii. With the inductive hypothesis, binarySearch(sortedArray, start, mid-1, value) works correctly as the size is less than n.
(5) If value is greater than sortedArray[mid]
    i. Prove as (4).

The running time of this algorithm is O(logn) as we continue cutting half of the array as illustrated below.

The sorted array: 0, 1, 2, 3, 4, 5, 6, 7, 8

Search for 3:

Iteration	Start	End	Mid	isFound?
1	0	8	4	No
2	0	3	1	No
3	2	3	2	No
4	3	3	3	Yes

The worst case scenario for this algorithm is when we need to cut an array until we reach an array of which size is 1, which is start==end.

N -> floor (N/2) -> … -> 1

The maximum number of iterations is ceil ($\log_2(n)$).

Therefore, in the case of Steven, the maximum number of getting his snicker eaten is equal to the number of making false guess, which is floor ($\log_2(n)$) + 1, where n = 1000. Therefore, he will lose at most 9+1= 10 snickers and the least number he can eat the snicker is 20 – 10 = 10. Hence, Steven will always eat Snickers more than or equal to Howard.

(B) (5 POINTS) [CROWD COUNTING – THE CROWD-SOURCING WAY]

(I) SYNCHRONIZED CASE

ASSUMING THAT WE START WITH N STUDENTS IN THE LECTURE THEATRE, WILL THIS ALGORITHM TERMINATE?

The algorithm always terminates as shown below.

Proof: show that the algorithm can terminates with n students.

Base case: When there is only one student, the algorithm terminates successfully as there is only one student standing who can report the number.

Induction Step: Show that with n students, the algorithm can terminate.

Inductive Hypothesis: The algorithm always terminate with n' students, where 0 <= n' < n.

(1) Firstly, we have n students.
(2) There are p = floor(n/2) pairs.
(3) Therefore, in the first iteration, p people will sit down.
(4) Hence, the number of standing students now is n – p.
(5) According to the inductive hypothesis, the algorithm can now terminate as n-p < n.
(6) Therefore, the algorithm can terminate with n students.

HOW MANY SYNCHRONIZED PHASES (OF STEP 2 AND 3) WILL IT TAKE?

In each iteration, half of the students will sit down. Therefore, the number of students decreases as illustrated below:

N -> N - floor (N/2) -> N – floor((N - floor (N/2)) /2) … -> 1

Which takes ceil($\log_2$(N)) iterations. Therefore, there will be ceil($\log_2$(N)) synchronized phases.

Supposed that N is a very large number, there will be O (log n) phases.

WHAT IS THE FINAL NUMBER IN STEP 4?

The final number is the sum of all the numbers by the student as proved below.

Proof: the final number is the summation of all the numbers by the student.

Base case: When the number of students is one, the final number is the number that a student holds which is equal to the summation of all the numbers by the student.

Induction Step: The final number of K students is equal to the sum of all the K numbers, where 0 < K < n.

(1) Let n be the number of students.
(2) There are p = floor (n/2) pairs.
(3) After the first iteration, p people sit down. Hence, the number of students become n-p holding the addition of the student and his pair.
(4) The final number of this n-p students is equal to the summation of the numbers of these n-p students.
(5) Each of the n-p students has added the value of people who have already sit down. Therefore, the final number of this n-p students is equal to the summation of n students.
(6) Hence, the final number of this n students is equal to the summation of n students.

HOW MANY ADDITIONS WILL BE DONE IN ALL?

Because the final number is equal to the sum of all the numbers. Therefore, the number of additions required is *the number of students – 1* as illustrated below

(((A + b) + (c + d)) + ((e + f) + g))

Brackets show how the paring is done.

(II) UNSYNCHRONIZED CASE

ASSUMING THAT WE START WITH N STUDENTS IN THE LECTURE THEATRE, WILL THIS ALGORITHM TERMINATE?

The algorithm still terminates; although, now not all the pairs are sitting at the same time. Standing people always finding a new partner and the number of students will decrease one by one until the number of students is equal to 1.

HOW MANY PHASES (OF STEP 2 AND 3) WILL IT TAKE?

The number of step 2 and step 3 is now varied. For example, supposed that there is a person who is very fast at matching and adding. Therefore, this person starts adding his number with all other students. Therefore, it will take n-1 phases to finish this procedure. Hence, the complexity is O (n). (However, previously in the synchronized case, the complexity is O(log n).

Therefore, the best case is log n complexity; however, the worst case is n complexity.

WHAT IS THE FINAL NUMBER IN STEP 4?

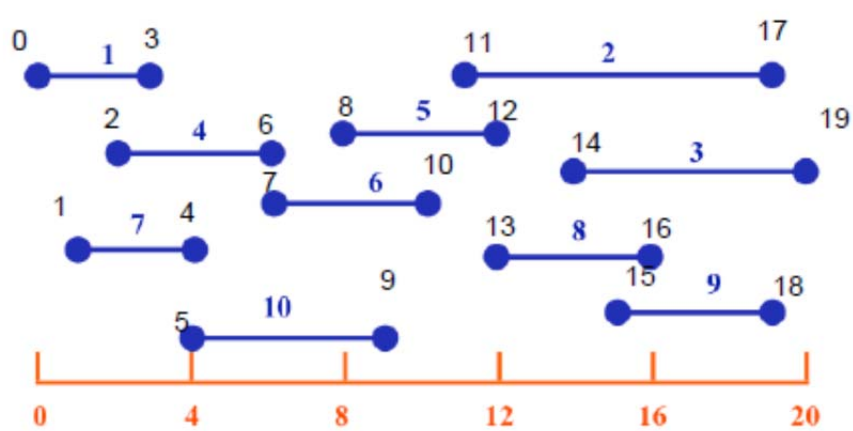
The final number is still the sum of all the students' number.

HOW MANY ADDITIONS WILL BE DONE IN ALL?

There is still going to be *the number of students – 1* because there are n numbers. To make a sum of all n numbers, we require n-1 additions.

S2 (10 POINTS) [ROOM ASSIGNMENT FOR ACTIVITIES]

(A)



k	T	A	Type	R1	R2	R3	R4
0	0	1	Start	1	-	-	-
1	1	7	Start	1	7	-	-

2	2	4	Start	1	7	4	-
3	3	1	End	-	7	4	-
4	3.9	7	End	-	-	4	-
5	4	10	Start	10	-	4	-
6	5.9	4	End	10	-	-	-
7	6	6	Start	10	6	-	-
8	8	5	Start	10	6	5	-
9	9	10	End	-	6	5	-
10	10	6	End	-	-	5	-
11	11	2	Start	2	-	5	-
12	11.9	5	End	2	-	-	-
13	12	8	Start	2	8	-	-
14	14	3	Start	2	8	3	-
15	15	9	Start	2	8	3	9
16	16	8	End	2	-	3	9
17	18.9	2	End	-	-	3	9
18	19	9	End	-	-	3	-
19	20	3	End	-	-	-	-

According to this algorithm, we can assign all the classes into four rooms.

Therefore, this assignment is optimal as we need at least 4 rooms at at t = 15.

(B)

Rewrite the algorithm as below for easy analysis.

Let T is a set of event points

(1) Sort T.
(2) Traverse T in an increasing order an applying case 1 and 2 as described in the question.

In (1), a sorting algorithm is required. There are multiple sorting algorithms and they have different time complexity and space complexity. For example, bubble sort has O (n^2) time complexity and O (1) space complexity. Merge sort takes O (n log n) time complexity and O (n) space complexity. Details are shown in the following table (from http://en.wikipedia.org/wiki/Sorting_algorithm#Comparison_of_algorithms).

Name	Best	Average	Worst	Memory	Method

Quicksort	n	$n \log n$	n^2	$\log n$ on average, worst case is n; Sedgewick variation is $\log n$ worst case	Partitioning
Merge sort	$n \log n$	$n \log n$	$n \log n$	n worst case	Merging
In-place merge sort	—	—	$n \log^2 n$	1	Merging
Heapsort	$n \log n$	$n \log n$	$n \log n$	1	Selection
Insertion sort	n	n^2	n^2	1	Insertion
Selection sort	n^2	n^2	n^2	1	Selection
Bubble sort	n	n^2	n^2	1	Exchanging

In the following code, I decide to use Java which uses merge sort or tuned quicksort. Therefore, in this implementation, step (1) takes O (n log n). (However, when the array is small, Java will change to insertion sort which is O (n^2). For simplicity, let's assume that Java always uses O (n log n) sorting algorithm.) (Reference: http://stackoverflow.com/questions/753237/what-sort-does-java-collections-sortnodes-use)

In (2), traversing an array takes O (2n), which is equal to O (n).

Therefore, overall complexity is O (n log n) + O (n) = O (n log n). The code I used to solve this problem can be seen in *Appendix A*. Below is the pseudocode showing the implementation.

Function ROOM_ASSIGNMENT(*EmptyRooms*, *EventPoints*) **returns** *TotalRooms*, *Room* with assignments

Inputs: *EmptyRooms*, an array of empty rooms that we can assign the event to.

EventPoints, an array of Event Point Object which has time, event type, and activity number.

Local Variables:

EmptyQueue, a queue of empty rooms to be assigned next, initially empty.

CurrentlyBooked, a hash map of an assignment and a room, initially empty.

RoomCount, an integer counting how many rooms are used, initially zero.

(step1) **SORT** *EventPoints* by using Merge sort/ Quick sort/ Heap sort

(step2) **for each** eventPoint **in** EventPoints:

```
            If eventPoint is a starting point then
                If EmptyQueue is empty then
                    Add EmptyRooms[RoomCount] to EmptyQueue
                    Increase RoomCount by 1
                Pop a room from an EmptyQueue into EmptyRoom variable.
                Add a mapping (eventPoint.activityNumber, emptyRoom) to CurrentlyBooked map
                Add an assignment (starting an eventPoint.activityNumber at eventPoint.time) to
emptyRoom
            Else
                room := CurrentlyBooked[eventPoint.activityNumber]
                Remove a mapping(eventPoint.activityNumber, room) from CurrentlyBooked map
                Add room to EmptyQueue
                Add an assignment (ending at eventPoint.activityNumber at eventPoint.time) to room.
            End for
        Returns RoomCount as TotalRooms, EmptyRooms.
```

S3

(A) (5 POINTS) [FINDING CYCLES IN LINEAR TIME]

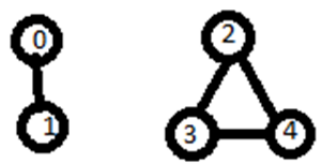
According to the definition of cycle, a cycle is "a circuit in which no vertex except the first (which is also the last) appears more than once." Therefore, a graph with no cycle should not have such a path that revisits a node because that path is a cycle.

The purposed algorithm to solve this problem is to traverse the graph, which can be done by either Breath-First Search (BFS) or Depth-First Search (DFS). While traversing, if there exists a node visited more than once, the graph contains a cycle and the algorithm can be terminated. In this case, DFS is implemented because it is more memory efficient than BFS. Below is an algorithm

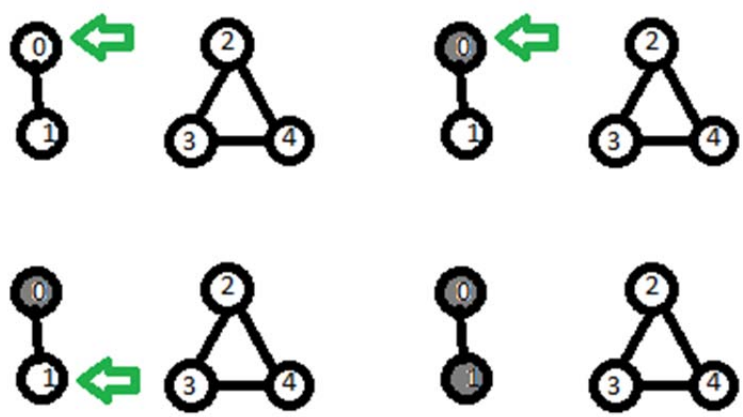
```
Function HAS_CYCLE(Graph G) returns a truth value
    Inputs: G, a graph G = (V, E) where V is a vertex list and E is an edge list with n vertices and m edges.
    Local variables:   IsVisited, a vector of Boolean values for all the vertices, initially false.
    If n=0 then return true
    For each v in V:
        If !IsVisited[v] and HAS_CYCLE_UTIL(v, IsVisited, null) then return true
    Return false

Function HAS_CYCLE_UTIL(Vertex v, Boolean[] IsVisited, Vertex parent) returns a truth value
    Inputs:   v, a vertex
```

```
        IsVisited, a vector of Boolean values for all the vertices
        Parent, a parent's vertex of v
    For each child in CHILDREN(v):
        If parent = child then continue
        If isVisited[child] = true then return true
        else
            isVisited[child] = true
            if HAS_CYCLE_UTIL(child, isVisited, parent) then return true
    Return false
```

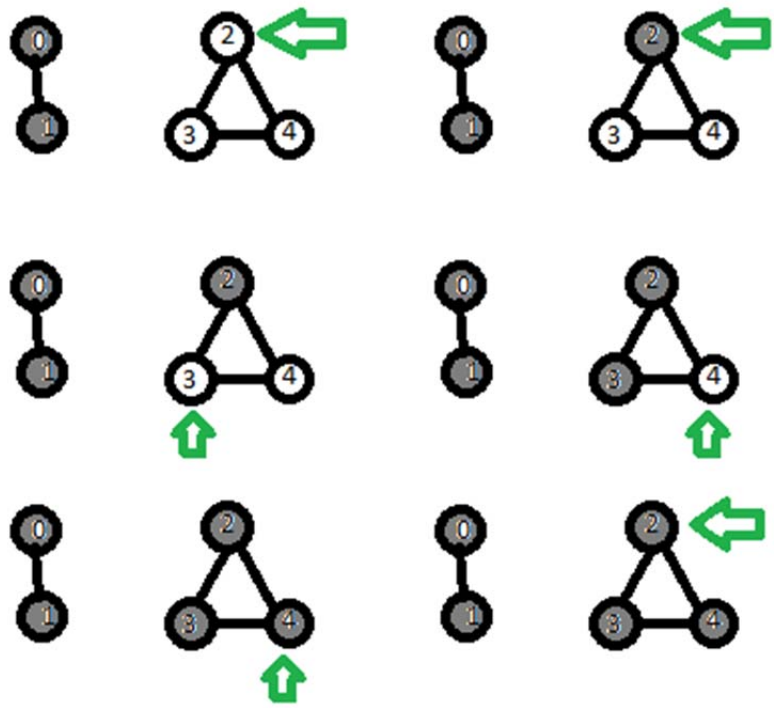
The following example shows how this algorithm works. A gray node means the node has already visited. A green arrow means the current Vertex v in HAS_CYCLE_UTIL.



Firstly, HAS_CYCLE_UTIL(0, isVisited, null) is called. DFS begins at 0.



Because now no cycle has been detected and IsVisited[0] and IsVisited[1] are now true and Visited[2] is false, HAS_CYCLE_UTIL(2, isVisited, null) is called next.



However, because when checking IsVisited of the child vertex 2 of the vertex 4, the vertex 2 has already been visited; hence, there exists a cycle in this graph. Therefore, true is returned from HAS_CYCLE_UTIL, resulting in true is retured from HAS_CYCLE.

The proof of correctness of this algorithm is shown below:

Proof There exists such a node that is visited more than once during DFS **IF AND ONLY IF** that graph is cyclic.

(1) If there exists such a node that is visited more than once, then the graph is cyclic. (A -> B)
   a. Proof by contradiction by proving that the graph is acyclic and there does not exist such a node that is visited more than once (A ^ not B) is not possible.
   b. Let a graph G is a cyclic graph that when the algorithm is executed, there does not exist such a node that is revisited.
   c. A graph G must contains a cycle C [u, v_1… v_n, u] (G is a cyclic graph).
   d. Once the DFS reach u for the first time, it will continue traversing until it reaches u again. Hence, there is a contradiction. (There does not exist such a node that is visited more than once.)

(2) If the graph is cyclic, then there exists a node that is visited more than once during DFS.
   a. Let a graph G be a cyclic graph with a cycle C [u, v_1…v_n, u]
   b. When DFS reaches u, DFS continues until it reaches u again. Hence, u is visited more than once.

The running time of DFS is O (|V|+|E|) because the first loop in HAS_CYCLE is executed at most O (|V|) and the loop inside HAS_CYCLE_UTIL is executed totally at most O(2|E|) because every vertex is run HAS_CYCLE only once and every edge is examined at most twice. So, for all HAS_CYCLE_UTIL, we have O (2|E|). Therefore, the time complexity is O (|V| + 2|E|) = O (|V| + |E|)

However, this algorithm has time complexity less than normal DFS because it has a possibility of terminating even before traversing all the vertices.

(1) Supposed that the graph has no cycle, the maximum number of edge is O (|V|-1). Therefore, the complexity becomes O (2|V| - 1) = O (|V|).

(2) Supposed that the graph has at least a cycle, there are more than O (|V| - 1). The algorithm will detect the cycle the most at O (|V|) edge. Therefore, the complexity is O (2|V|) = O (|V|)

Hence, time complexity of this algorithm is O (|V|).

(B) (5 POINTS) [MUTUAL-ATTRACTION IN STABLE MARRIAGE]

Proof: In every stable matching S for this instance, the pair (m*, w+) belongs to S.

(Proof by contradiction)

(1) Let *S* be a stable matching that does not have the pair *(m*, w+)*
(2) Therefore, *m** never proposed to *w+* or *w+* rejected *m** after being proposed.
(3) Case 1: *m** never proposed to *w+.*
    a. *m** prefers his girlfriend to *w+.*
    b. His girlfriend has higher rank than *w+.*
    c. This is a contradiction.
(4) Case 2*: m** is rejected by *w+*
    a. *w+* prefers her boyfriend to *m*.*
    b. Her boyfriend has higher rank than *m**
    c. This is a contradiction.

APPENDIX A

```
import java.util.ArrayList;
import java.util.Collections;
import java.util.HashMap;
import java.util.LinkedList;
import java.util.Queue;

public class LineSweep {
        HashMap<Activity, Room> assignments;
        Queue<Room> emptyRooms;
        ArrayList<Room> rooms;

        public static void main(String[] args) {
                ArrayList<T> t = initProblem();

                LineSweep ls = new LineSweep();
```

```java
            ls.assignRoom(t);
            ArrayList<Room> rooms = ls.getRooms();

            for (Room r : rooms) {
                    r.printAssignment();
            }
    }

    private static ArrayList<T> initProblem() {
            ArrayList<Activity> activities = initializeActivities();
            ArrayList<T> t = initializeT(activities);
            return t;
    }

    private static ArrayList<Activity> initializeActivities() {
            ArrayList<Activity> activities = new ArrayList<Activity>();
            activities.add(new Activity(0)); // dummy
            for (int i = 1; i <= 10; i++)
                    activities.add(new Activity(i));
            return activities;
    }

    private static ArrayList<T> initializeT(ArrayList<Activity> activities) {
            ArrayList<T> t = new ArrayList<T>();
            t.add(new T(activities.get(1), "START", 0));
            t.add(new T(activities.get(1), "END", 3));

            t.add(new T(activities.get(2), "START", 11));
            t.add(new T(activities.get(2), "END", 19));

            t.add(new T(activities.get(3), "START", 14));
            t.add(new T(activities.get(3), "END", 20));

            t.add(new T(activities.get(4), "START", 2));
            t.add(new T(activities.get(4), "END", 6));

            t.add(new T(activities.get(5), "START", 8));
            t.add(new T(activities.get(5), "END", 12));

            t.add(new T(activities.get(6), "START", 6));
            t.add(new T(activities.get(6), "END", 10));

            t.add(new T(activities.get(7), "START", 1));
            t.add(new T(activities.get(7), "END", 4));

            t.add(new T(activities.get(8), "START", 12));
            t.add(new T(activities.get(8), "END", 16));

            t.add(new T(activities.get(9), "START", 15));
```

```java
                t.add(new T(activities.get(9), "END", 19));

                t.add(new T(activities.get(10), "START", 4));
                t.add(new T(activities.get(10), "END", 9));
                return t;
        }

        private ArrayList<Room> getRooms() {
                return rooms;
        }

        public void assignRoom(ArrayList<T> T_list) {
                assignments = new HashMap<Activity, Room>();
                emptyRooms = new LinkedList<Room>();
                rooms = new ArrayList<Room>();

                Collections.sort(T_list); // O(n log n) operation.
                for (T t : T_list) {        // O( 2 n )
                        if (t.isStarting()) {
                                if (emptyRooms.isEmpty()) {
                                        Room room = new Room();
                                        rooms.add(room);
                                        emptyRooms.add(room);
                                }
                                Room emptyRoom = emptyRooms.poll();
                                emptyRoom.add(t);
                                assignments.put(t.activity, emptyRoom);
                        } else {
                                Room previouslyOccupiedRoom = assignments.get(t.activity);
                                previouslyOccupiedRoom.add(t);
                                assignments.remove(t.activity);
                                emptyRooms.add(previouslyOccupiedRoom);
                        }
                }
        }
}

class T implements Comparable<T> {
        Double time;
        String type;
        Activity activity;

        public boolean isStarting() {
                return type == "START";
        }

        public boolean isEnding() {
                return !isStarting();
        }
```

```java
	public T(Activity a, String type, double t) {
		this.activity = a;
		this.type = type;
		this.time = t;
	}

	@Override
	public int compareTo(T another) {
		if (time.compareTo(another.time) != 0)
			return time.compareTo(another.time);
		else
			return isEnding() ? -1 : 1; // if it is ending, should try to free
								// the room first.
	}
}

class Activity {
	int number;

	public Activity(int i) {
		this.number = i;
	}

	@Override
	public int hashCode() {
		return number;
	}

	@Override
	public boolean equals(Object o) {
		if (o instanceof Activity)
			return this.number == ((Activity) o).number;
		return false;
	}
}

class Room {
	int number;
	ArrayList<T> assignments = new ArrayList<T>();

	@Override
	public int hashCode() {
		return number;
	}

	public void printAssignment() {
		StringBuilder output = new StringBuilder();
		for (T assignment : assignments) {
```

```
                        output.append(assignment.activity.number).append(' ')
                                        .append(assignment.type).append(' ')
                                        .append(assignment.time).append('\n');
                }
                System.out.println(output.toString());
        }

        public void add(T t) {
                assignments.add(t);
        }

        @Override
        public boolean equals(Object o) {
                if (o instanceof Activity)
                        return this.number == ((Activity) o).number;
                return false;
        }
}
```

Output of this program from the defined problem is shown below. Each room assignment is separated by an empty line. Hence, there are four rooms in total as seen below.

```
1 START 0
1 END 3
10 START 4
10 END 9
2 START 11
2 END 18.9

7 START 1
7 END 3.9
6 START 6
6 END 10
8 START 12
8 END 16

4 START 2
4 END 5.9
5 START 8
5 END 11.9
3 START 14
3 END 20

9 START 15
9 END 19
```